Software Requirements Specification

for

Airport Management System

Version 1.0 approved

Prepared by Adwait and Jovin

MCT's Rajiv Gandhi Institute of Technology (RGIT), Mumbai

Department of Computer Engineering

14 August 2026

IEEE/ISO/IEC 29148:2018-aligned Software Requirements Specification
Prepared using the supplied SRS template structure.

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **ii**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements **4**
3.1 User Interfaces 4
3.2 Hardware Interfaces 4
3.3 Software Interfaces 4
3.4 Communications Interfaces 4
4. System Features **5**
4.1 Authentication and Access Control 5
4.2 Flight Management 5
4.3 Booking Module 6
4.4 Passenger Module 7
4.5 Employee Module 7
4.6 Baggage Module 8
4.7 Gate Management and Boarding 8
4.8 Dashboard and Reporting 9
5. Other Nonfunctional Requirements **10**
5.1 Performance Requirements 10
5.2 Safety Requirements 10
5.3 Security Requirements 10
5.4 Software Quality Attributes 10
5.5 Business Rules 11
6. Other Requirements **11**
6.1 Database Requirements 11
6.2 REST API Summary 12
6.3 Internationalization Requirements 13
6.4 Legal / Academic Requirements 13

Appendix A: Glossary 14
Appendix B: Analysis Models 15
Appendix C: To Be Determined List 15

Revision History

Name	Date	Reason For Changes	Version
RGIT AMS Project Team	13 Aug 2026	Initial draft of SRS created.	1.0
RGIT AMS Project Team	14 Aug 2026	SRS restructured to IEEE/ISO/IEC 29148:2018-aligned format (REQ-Fxx / REQ-NFxx numbering); structure retained from supplied template.	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, interface, performance, security, quality, and operational requirements of the Airport Management System (AMS), Version 1.0. The system is intended to support the digitization of day-to-day airport operations by providing structured storage of flight, passenger, and staff information, ticket booking and check-in facilities, gate and baggage tracking, and a centralized operational dashboard for all four supported user roles: Admin, Airline Staff, Passenger, and Ground Staff.

1.2 Document Conventions

The document follows the section organization of the supplied SRS template and uses requirement statements that are intended to be clear, necessary, unambiguous, verifiable, and uniquely identifiable. Requirement identifiers use the format REQ-Fxx for functional requirements and REQ-NFxx for nonfunctional requirements. The word shall denotes a mandatory requirement; should denotes a recommended requirement. Priority is identified as High, Medium, or Low where applicable. Database table and field names are written in fixed-width style (e.g., flight_id) to distinguish them from ordinary text. The keyword TBD is used as a placeholder wherever information is not yet finalized; all TBD items are consolidated in Appendix C.

1.3 Intended Audience and Reading Suggestions

The intended audience includes the project guide/evaluators, developers (frontend, backend, database), testers/QA, and project team members. Readers who need project context should begin with Sections 1 and 2. Developers and testers should then review Sections 3, 4, and 5, which are the most technically significant sections. Section 5 should be reviewed before design sign-off, and the appendices may be referred to as needed.

1.4 Product Scope

The Airport Management System is a full-stack, role-based web application built to replace manual and fragmented airport administrative processes with a single, centralized digital platform. The system enables administrators to manage airlines, airports, flights, gates, and employees; allows airline staff to schedule flights and manage passenger manifests; lets passengers search, book, and manage their own flight tickets and boarding passes; and supports ground staff in gate assignment, check-in verification, and baggage tracking. The primary objectives of the system are to reduce manual errors, improve operational visibility through a live dashboard, and provide passengers with a self-service booking and check-in experience. The project is developed as an academic capstone project using the MERN-adjacent stack (React.js, Node.js, Express.js, MySQL) and is not intended for production deployment at a real airport without further security hardening and regulatory compliance review.

1.5 References

- IEEE/ISO/IEC 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
- IEEE 830-1998, IEEE Recommended Practice for Software Requirements Specifications, used historically for SRS structure; it has been superseded by ISO/IEC/IEEE 29148.
- Wiegers, Karl E. — Software Requirements Specification Template, 1999 (base document format used for this SRS).
- React.js Official Documentation — https://react.dev
- Node.js Official Documentation — https://nodejs.org/en/docs
- Express.js Official Documentation — https://expressjs.com
- MySQL 8.0 Reference Manual — https://dev.mysql.com/doc/
- JSON Web Token (JWT) — RFC 7519, https://datatracker.ietf.org/doc/html/rfc7519

2. Overall Description

2.1 Product Perspective

The Airport Management System is a new, self-contained web application and is not a follow-on to any existing system. It consists of three major components: a React.js single-page frontend, a Node.js/Express.js REST API backend, and a MySQL relational database. The frontend communicates with the backend exclusively through documented REST APIs secured using JWT-based authentication. The system is designed as a monolithic full-stack application for the scope of this project, with the backend and database deployed independently of the frontend (frontend on Netlify/Vercel; backend and database on Render), communicating over HTTPS. At a high level, the system consists of a Presentation Layer (React.js, Tailwind CSS, React Router), an Application/API Layer (Node.js, Express.js), a Data Layer (MySQL, 11 relational tables), and simulated external services (mock payment gateway, simulated live flight tracking, and optionally an AI chatbot / delay-prediction service).

2.2 Product Functions

- Authenticate users and apply role-based access permissions for Admin, Airline Staff, Passenger, and Ground Staff.
- Maintain airline, airport, aircraft, gate, and employee master data.
- Schedule, update, and search flights, and assign aircraft.
- Allow passengers to search flights, select seats, book and cancel tickets, and make mock payments.
- Generate and deliver boarding passes following verified check-in.
- Track checked baggage status and support lost-baggage reporting.
- Assign gates to flights and manage the boarding process.
- Provide a live operations dashboard summarizing total flights, delayed flights, active passengers, revenue, and gate availability.

2.3 User Classes and Characteristics

User Class	Characteristics / Importance
Admin	Full system access; manages master data (airlines, airports, gates, employees) and views all reports. Assumed to be a trained, technically comfortable staff member. Most important — core operational control.
Airline Staff	Manages flights and passenger manifests for their airline. Moderate technical expertise; uses the system daily during working hours. Most important — primary data entry role.
Passenger	General public with varying technical expertise, using the system occasionally to book or manage travel. Requires a simple, guided UI. Most important — primary revenue-generating user.
Ground Staff	Operational staff working at gates/check-in counters; needs a fast, low-friction interface, often used on shared terminals. Important — time-critical operations.

2.4 Operating Environment

- Client: any modern web browser (Chrome, Firefox, Edge, Safari — latest two versions) on desktop or mobile, supporting a responsive layout built with Tailwind CSS.
- Frontend hosting: static React build deployed on Netlify or Vercel.
- Backend hosting: Node.js/Express.js REST API deployed on Render.
- Database: MySQL 8.x, hosted on Render (or an equivalent managed MySQL instance).
- Communication: all client–server communication occurs over HTTPS; the frontend and backend are hosted as separate services and communicate via CORS-enabled REST APIs.

2.5 Design and Implementation Constraints

- The technology stack is fixed for this project: React.js, Tailwind CSS, React Router (frontend); Node.js, Express.js (backend); MySQL (database); JWT and bcrypt (authentication).
- Payments are simulated (mock payment gateway) — no real financial transactions or PCI-DSS compliance are implemented in this version.
- The database schema must follow relational design principles and be normalized to at least 3NF (Third Normal Form).
- Live flight tracking and delay prediction are simulated/rule-based or use a basic ML model trained on limited historical data; they are not connected to real-world aviation data feeds (e.g., FlightAware, ADS-B).
- The system is being built as an academic capstone project within a limited timeframe using an Agile sprint structure, which constrains the depth of features delivered in each sprint.
- Role-based access control (RBAC) must be enforced at the API level using JWT claims, not solely in the frontend UI.

2.6 User Documentation

- An in-app onboarding guide / tooltip help for first-time users of each role.
- A README file describing setup, environment variables, and deployment steps for developers.
- A short user manual (PDF) covering the passenger booking flow and the admin dashboard, delivered as part of the final project report.

2.7 Assumptions and Dependencies

- It is assumed that users have access to a stable internet connection and a modern web browser.
- It is assumed that airline, airport, and aircraft master data will be seeded/entered by the Admin before flight scheduling begins.
- The project depends on third-party npm packages (e.g., jsonwebtoken, bcrypt, mysql2, react-router-dom) remaining available and compatible with the chosen Node.js version.
- The project depends on free-tier hosting availability on Netlify/Vercel and Render for deployment; any changes to these platforms' free-tier limits could affect availability.
- It is assumed that QR-code boarding passes and email notifications (advanced features) rely on third-party libraries/services that are assumed to be freely available for this project's scale.

3. External Interface Requirements

3.1 User Interfaces

The system provides four distinct, role-based web interfaces, all built with React.js and styled using Tailwind CSS for a responsive, mobile-friendly layout. Data-entry screens shall identify mandatory fields and display validation messages when input is missing, invalid, duplicated, or inconsistent.

- Common: a shared Login/Signup screen with a 'Forgot Password' link; a persistent navigation bar reflecting the logged-in user's role; consistent error message styling (red banner with a short, human-readable message) across all screens.
- Admin Dashboard: sidebar navigation to Airlines, Airports, Flights, Gates, Employees, and Reports; data tables with search/filter and CRUD action buttons; chart widgets for the analytics dashboard.
- Airline Staff Console: flight scheduling forms, an editable passenger manifest table, and a boarding-pass generation action.
- Passenger Portal: a flight search form (origin, destination, date), a seat-map seat-selection component, a booking checkout flow with a mock payment step, and a 'My Bookings' history page with a downloadable boarding pass.
- Ground Staff Console: a simplified, high-contrast interface optimized for quick actions — gate assignment, check-in verification (by booking ID / QR scan), and baggage status update.

Keyboard shortcuts are not required for Version 1.0. All screens shall include a visible 'Help' icon linking to the onboarding guide referenced in Section 2.6.

3.2 Hardware Interfaces

The system has no direct hardware interfaces in Version 1.0. It is a purely web-based application accessed through standard client hardware (desktop, laptop, tablet, or smartphone) with a display and internet connectivity. An optional future hardware interface — a physical QR/barcode scanner at check-in counters — is noted as TBD in Appendix C.

3.3 Software Interfaces

The application shall interface with React.js (18.x) for frontend UI rendering and component-based state management, React Router (6.x) for client-side routing, and Tailwind CSS (3.x) for utility-first responsive styling. The backend shall use Node.js (18.x LTS) as the JavaScript runtime and Express.js (4.x) for REST API routing and middleware (auth, validation, error handling). Persistent storage shall use MySQL (8.x) across all 11 core tables (see Section 6.1). Authenticated sessions shall use JWT (jsonwebtoken 9.x) for issuing and verifying signed access tokens, and bcrypt (5.x) for one-way hashing of user passwords before storage. All data exchanged between the frontend and backend shall be formatted as JSON over HTTP(S), following the REST API contract defined in Section 6.2.

3.4 Communications Interfaces

- Protocol: all client–server communication uses HTTPS (TLS) with RESTful JSON payloads.
- Authentication headers: authenticated requests shall include an Authorization: Bearer <JWT> header; the token is issued at login and validated on every protected route.
- CORS: the backend enables Cross-Origin Resource Sharing restricted to the deployed frontend origin(s).
- Email notifications (advanced feature): outbound transactional email (e.g., booking confirmation) is sent via a third-party SMTP/email API; delivery is best-effort and not guaranteed in real time.
- No real-time socket/streaming protocol is required for Version 1.0; simulated live flight tracking is implemented via periodic REST polling rather than WebSockets.

4. System Features

Functional requirements are organized by System Feature, corresponding to the modules listed in Section 2.2. Each feature includes a description and priority, representative stimulus/response sequences, and itemized, uniquely identified functional requirements using the REQ-Fxx convention defined in Section 1.2.

4.1 Authentication and Access Control

4.1.1 Description and Priority

The system shall provide secure registration, login, password recovery, and role-based access control (RBAC) for all four user classes. Priority: High.

4.1.2 Stimulus/Response Sequences

- User submits the signup form with valid details → the system creates the account with a hashed password and assigns a default role.
- User submits valid login credentials → the system returns a signed JWT and redirects to the role-appropriate dashboard.
- User requests 'Forgot Password' → the system sends a reset link/OTP and allows setting a new password.
- Unauthorized user attempts to access a role-restricted route → the system returns a 403 Forbidden response and redirects to an error page.

4.1.3 Functional Requirements

REQ-F01: The system shall allow a new user to register with name, email, password, and role.

REQ-F02: The system shall hash all passwords using bcrypt before persisting them to the users table; plaintext passwords shall never be stored.

REQ-F03: The system shall authenticate users via email and password and issue a signed JWT upon successful login.

REQ-F04: The system shall reject login attempts with invalid credentials and display a generic error message without revealing whether the email or password was incorrect.

REQ-F05: The system shall enforce role-based access control on every protected API endpoint by validating the role claim embedded in the JWT.

REQ-F06: The system shall provide a 'Forgot Password' flow that allows a user to reset their password via a verified email address.

REQ-F07: The system shall expire JWTs after a configurable duration (TBD-01) and require re-authentication thereafter.

4.2 Flight Management

4.2.1 Description and Priority

The system shall allow Admins and Airline Staff to create, update, delete, search, and schedule flights, including assigning airlines and aircraft. Priority: High.

4.2.2 Stimulus/Response Sequences

- Airline Staff submits a new flight form (route, times, aircraft) → the system validates and creates a flight record with status 'Scheduled'.
- Admin/Airline Staff updates flight departure time → the system updates the record and, in the advanced version, triggers a passenger notification.
- Passenger or staff searches flights by origin, destination, and date → the system returns matching, filterable results.

4.2.3 Functional Requirements

REQ-F08: The system shall allow Admin and Airline Staff to add a new flight with airline, aircraft, departure/arrival airports, and departure/arrival times.

REQ-F09: The system shall allow Admin and Airline Staff to update an existing flight's schedule, aircraft assignment, or status (Scheduled, Delayed, Boarding, Departed, Cancelled, Arrived).

REQ-F10: The system shall allow Admin to delete a flight only if it has no active (non-cancelled) bookings.

REQ-F11: The system shall allow any user to search flights by origin, destination, and date, with results filterable by airline and status.

REQ-F12: The system shall prevent assigning an aircraft to two overlapping flights (double-scheduling).

REQ-F13: The system shall validate that arrival time is later than departure time when a flight is created or updated.

4.3 Booking Module

4.3.1 Description and Priority

The system shall enable passengers to search flights, select seats, book tickets, make a (mock) payment, and cancel bookings. Priority: High.

4.3.2 Stimulus/Response Sequences

- Passenger selects a flight and an available seat → the system reserves the seat temporarily during checkout.
- Passenger completes the mock payment step → the system confirms the booking, generates a booking record, and updates seat availability.
- Passenger requests cancellation of a booking → the system updates booking_status to 'Cancelled' and releases the seat.

4.3.3 Functional Requirements

REQ-F14: The system shall display real-time seat availability for a selected flight based on the aircraft's capacity and existing bookings.

REQ-F15: The system shall allow a passenger to select an available seat and proceed to checkout.

REQ-F16: The system shall process a mock payment and create a corresponding record in the payments table with payment_status.

REQ-F17: The system shall create a booking record only after successful (mock) payment confirmation.

REQ-F18: The system shall prevent two passengers from being assigned the same seat_no on the same flight_id.

REQ-F19: The system shall allow a passenger to cancel a booking prior to a configurable cutoff time (TBD-02) before departure and shall release the seat upon cancellation.

REQ-F20: The system shall display the passenger's complete booking history, including cancelled bookings.

4.4 Passenger Module

4.4.1 Description and Priority

The system shall provide passengers with profile management, booking history, boarding pass access, and status notifications. Priority: Medium.

4.4.2 Stimulus/Response Sequences

- Passenger views 'My Bookings' → the system lists all bookings with current status.
- Passenger requests a boarding pass for a confirmed booking → the system generates and displays a downloadable boarding pass.
- Flight status changes (e.g., delayed) → the system displays an in-app notification to affected passengers.

4.4.3 Functional Requirements

REQ-F21: The system shall allow a passenger to view and update their profile information (name, phone, passport number).

REQ-F22: The system shall generate a boarding pass (pass_id, gate, boarding_time) once check-in is completed (see Section 4.7).

REQ-F23: The system shall allow a passenger to download their boarding pass as a PDF.

REQ-F24: The system shall display in-app notifications to a passenger when the status of a booked flight changes.

REQ-F25: The system shall allow a passenger to track the real-time (simulated) status of any of their booked flights.

4.5 Employee Module

4.5.1 Description and Priority

The system shall allow Admin to manage employee records, departments, designations, attendance, and shift schedules for airline and ground staff. Priority: Medium.

4.5.2 Stimulus/Response Sequences

- Admin adds a new employee record with department and designation → the system creates the employee record linked to a user account.
- Admin assigns a shift to an employee → the system updates the employee's shift schedule.

4.5.3 Functional Requirements

REQ-F26: The system shall allow Admin to create, update, and deactivate employee records with department and designation.

REQ-F27: The system shall allow Admin to record and view daily attendance for each employee.

REQ-F28: The system shall allow Admin to assign and update shift schedules for employees.

REQ-F29: The system shall restrict employee-record management actions to users with the Admin role.

4.6 Baggage Module

4.6.1 Description and Priority

The system shall enable Ground Staff to track checked baggage, update its status, and log lost-baggage reports linked to a booking. Priority: Medium.

4.6.2 Stimulus/Response Sequences

- Ground Staff checks in baggage for a booking → the system creates a baggage record with weight and status 'Checked-In'.
- Ground Staff updates baggage status (e.g., 'Loaded', 'Delivered', 'Lost') → the system updates the baggage record and, for passengers, the linked notification.

4.6.3 Functional Requirements

REQ-F30: The system shall allow Ground Staff to register baggage against a booking_id with recorded weight.

REQ-F31: The system shall allow Ground Staff to update baggage status through defined states (Checked-In, Loaded, In-Transit, Delivered, Lost).

REQ-F32: The system shall allow a passenger or Ground Staff to file a lost-baggage report against a baggage_id.

REQ-F33: The system shall display current baggage status to the passenger on their booking details page.

4.7 Gate Management and Boarding

4.7.1 Description and Priority

The system shall enable Ground Staff and Admin to assign gates to flights, verify passenger check-in, and manage the boarding process. Priority: Medium.

4.7.2 Stimulus/Response Sequences

- Admin/Ground Staff assigns an available gate to a flight → the system updates the gate's status and links it to the flight.
- Passenger checks in at the counter or digitally → Ground Staff verifies the booking and issues/confirms the boarding pass.

4.7.3 Functional Requirements

REQ-F34: The system shall allow Admin/Ground Staff to assign an available gate to a scheduled flight.

REQ-F35: The system shall prevent assigning an already-occupied gate to a conflicting flight at an overlapping time.

REQ-F36: The system shall allow Ground Staff to verify passenger check-in by booking ID or scanned QR code.

REQ-F37: The system shall generate a boarding pass with gate and boarding_time only after successful check-in verification.

REQ-F38: In the advanced version, the system should suggest a gate assignment based on aircraft size and gate availability (smart gate assignment).

4.8 Dashboard and Reporting

4.8.1 Description and Priority

The system shall provide Admin with a real-time analytics dashboard summarizing key operational metrics. Priority: Medium.

4.8.2 Stimulus/Response Sequences

- Admin opens the Dashboard → the system fetches and displays aggregated metrics and charts.
- Admin applies a date-range filter → the system recalculates and redisplays the metrics for the selected range.

4.8.3 Functional Requirements

REQ-F39: The system shall display total flights, delayed flights, active passengers, total revenue, and available gates on the Admin dashboard.

REQ-F40: The system shall render dashboard metrics as charts (e.g., bar/line/pie) reflecting current data.

REQ-F41: The system shall allow the Admin to filter dashboard metrics by a selected date range.

REQ-F42: The system shall refresh dashboard data on demand and/or at a periodic interval (TBD-03).

5. Other Nonfunctional Requirements

5.1 Performance Requirements

REQ-NF01: The system shall return flight search results within 2 seconds under normal load (up to 100 concurrent users), to keep the booking flow responsive.

REQ-NF02: The system shall support at least 500 registered passenger accounts and 50 concurrent bookings without data inconsistency, sufficient for the demonstration scale of this academic project.

REQ-NF03: Dashboard queries shall complete within 3 seconds for datasets of up to 10,000 booking records.

5.2 Safety Requirements

REQ-NF04: The system shall prevent double-booking of the same seat on the same flight and double-assignment of the same gate to overlapping flights, since these directly reflect real-world safety-relevant scheduling constraints.

REQ-NF05: The system shall not permit deletion of a flight, aircraft, or airport record that has dependent active bookings, to avoid orphaned or inconsistent operational data.

REQ-NF06: Since this is an academic simulation, no life-safety-critical hardware (e.g., actual air-traffic control systems) is interfaced with; the safety requirements above are data-integrity safeguards, not physical safety systems.

5.3 Security Requirements

REQ-NF07: All passwords shall be hashed using bcrypt; plaintext passwords shall never be logged or stored (see REQ-F02).

REQ-NF08: All API endpoints that modify or expose user, booking, or payment data shall require a valid JWT and shall enforce role-based authorization.

REQ-NF09: All client–server traffic shall be encrypted using HTTPS/TLS.

REQ-NF10: Sensitive fields (e.g., passport_no) shall be accessible only to the owning passenger and to Admin/Airline Staff roles that require them for operational purposes.

REQ-NF11: The mock payment module shall not collect or store real card/financial details, since no PCI-DSS-compliant payment processing is implemented in this version.

5.4 Software Quality Attributes

REQ-NF12: Usability: a first-time passenger shall be able to complete a flight booking within 5 steps without external help, supported by the onboarding guide (Section 2.6).

REQ-NF13: Reliability: the booking and payment workflow shall maintain data consistency even if a request fails mid-transaction (i.e., no booking shall be created without a corresponding successful payment record).

REQ-NF14: Maintainability: backend code shall be organized by module (routes/controllers per feature) to allow independent modification of one feature without affecting others.

REQ-NF15: Portability: the frontend shall render correctly on the latest two versions of Chrome, Firefox, Edge, and Safari, and shall be responsive down to a 375px mobile viewport width.

REQ-NF16: Availability: the deployed system shall target at least 95% uptime during the evaluation/demo period, subject to free-tier hosting limitations.

5.5 Business Rules

- Only users with the Admin role may create, update, or delete airline, airport, aircraft, and employee master data.
- Only Airline Staff associated with a given airline_id may schedule or modify flights for that airline.
- A booking may be cancelled by the passenger only before the cutoff defined in REQ-F19; cancellations after this point must be handled manually by Airline Staff.

6. Other Requirements

The system shall maintain persistent records for users, master data, bookings, and operational status across a normalized MySQL relational database (target: 3NF) comprising 11 core tables. Primary and foreign key relationships enforce referential integrity between passengers/employees and their linked user accounts, between flights and their airline/aircraft, and between bookings and their flight/passenger/payment/baggage/boarding-pass records.

6.1 Database Requirements

Users

Field	Description
id	Primary key; unique user identifier
name	Full name of the user
email	Unique login email address
password	bcrypt-hashed password
role	One of: Admin, Airline Staff, Passenger, Ground Staff

Passengers

Field	Description
passenger_id	Primary key; foreign key to Users.id
passport_no	Passenger's passport number
phone	Contact phone number

Employees

Field	Description
employee_id	Primary key; foreign key to Users.id
department	Department the employee belongs to
designation	Job title / designation

Airlines

Field	Description
airline_id	Primary key; unique airline identifier
airline_name	Name of the airline

Aircraft

Field	Description
aircraft_id	Primary key; unique aircraft identifier
airline_id	Foreign key to Airlines.airline_id
model	Aircraft model
capacity	Total seating capacity

Flights

Field	Description
flight_id	Primary key; unique flight identifier
airline_id	Foreign key to Airlines.airline_id
aircraft_id	Foreign key to Aircraft.aircraft_id
departure	Departure airport
arrival	Arrival airport
departure_time	Scheduled departure date/time
arrival_time	Scheduled arrival date/time
status	Scheduled / Delayed / Boarding / Departed / Cancelled / Arrived

Airports

Field	Description
airport_id	Primary key; unique airport identifier
airport_name	Name of the airport
city	City where the airport is located
country	Country where the airport is located

Gates

Field	Description
gate_id	Primary key; unique gate identifier
airport_id	Foreign key to Airports.airport_id
status	Available / Occupied / Under Maintenance

Bookings

Field	Description
booking_id	Primary key; unique booking identifier
passenger_id	Foreign key to Passengers.passenger_id
flight_id	Foreign key to Flights.flight_id
seat_no	Assigned seat number
booking_status	Confirmed / Cancelled / Pending

BoardingPass

Field	Description
pass_id	Primary key; unique boarding pass identifier
booking_id	Foreign key to Bookings.booking_id
gate	Assigned boarding gate
boarding_time	Scheduled boarding time

Baggage

Field	Description
baggage_id	Primary key; unique baggage identifier
booking_id	Foreign key to Bookings.booking_id
weight	Baggage weight (kg)
status	Checked-In / Loaded / In-Transit / Delivered / Lost

Payments

Field	Description
payment_id	Primary key; unique payment identifier
booking_id	Foreign key to Bookings.booking_id
amount	Payment amount
payment_status	Success / Failed / Pending (mock payment gateway)

- The database shall uniquely identify records that require unique identification.
- Relationships between bookings and referenced master data shall be enforced.
- The database shall prevent invalid references to deleted or nonexistent master records.
- The system should support backup and recovery procedures appropriate to the deployment environment.

6.2 REST API Summary

Method / Endpoint	Purpose
POST /login	Authenticate a user and issue a JWT
POST /signup	Register a new user account
GET /flights	Search / list flights
POST /flights	Create a new flight (Admin / Airline Staff)
PUT /flights/:id	Update an existing flight
DELETE /flights/:id	Delete a flight (subject to REQ-F10)
POST /book	Create a new booking
GET /booking/:id	Retrieve a specific booking
DELETE /booking/:id	Cancel a booking
GET /boarding-pass/:id	Retrieve a boarding pass
POST /checkin	Verify passenger check-in
GET /baggage	Retrieve baggage status
POST /payment	Process a (mock) payment
GET /dashboard	Retrieve aggregated dashboard metrics

6.3 Internationalization Requirements

A multi-language interface is identified as a stand-out (advanced) feature and is not a mandatory requirement for Version 1.0. If implemented, all user-facing strings shall be externalized into locale resource files to support future language additions without code changes (TBD-04).

6.4 Legal / Academic Requirements

This system is developed as an academic capstone/mini-project and is not intended for commercial deployment. No real passenger financial or passport data shall be used during development, testing, or demonstration; only synthetic/sample data shall be used.

Appendix A: Glossary

Term	Definition
AMS	Airport Management System — the product described in this SRS.
JWT	JSON Web Token; a signed token used to authenticate API requests.
RBAC	Role-Based Access Control; restricting system actions based on a user's assigned role.
REST	Representational State Transfer; the architectural style used for the API.
SRS	Software Requirements Specification; this document.
3NF	Third Normal Form; a level of database normalization that reduces data redundancy.
ERD	Entity-Relationship Diagram; a diagram modeling the database schema and relationships.
Mock Payment	A simulated payment step that mimics a real payment gateway without processing actual financial transactions.
TBD	To Be Determined; a placeholder for information not yet finalized.

Appendix B: Analysis Models

The following analysis models are recommended as part of the project design documentation. They are referenced here because the SRS defines the information and interactions that those models should represent; actual diagrams should be created using a UML tool (e.g., draw.io, Lucidchart, or StarUML) and maintained as separate artifacts alongside the project report.

B.1 Context Model

The context model should show the Airport Management System as the central process and the primary external actors: Admin, Airline Staff, Passenger, and Ground Staff. Inputs include master data, flight schedules, booking requests, and check-in/baggage updates. Outputs include confirmed bookings, boarding passes, gate assignments, and dashboard metrics.

B.2 Use-Case Model

- Authenticate User
- Manage Airlines, Airports, Aircraft, Gates, and Employees
- Schedule / Update Flight
- Search Flights
- Book / Cancel Ticket
- Process Mock Payment
- Check In Passenger
- Generate Boarding Pass
- Assign Gate

- Track / Update Baggage Status
- View Dashboard and Reports

B.3 Entity-Relationship Model

The ER model should represent at minimum the entities User, Passenger, Employee, Airline, Aircraft, Flight, Airport, Gate, Booking, BoardingPass, Baggage, and Payment, corresponding to the 11 database tables listed in Section 6.1. Relationships should ensure that each booking references valid flight and passenger data and that mandatory uniqueness/conflict rules (seat and gate double-assignment) can be checked.

B.4 Data Flow Model

A high-level data-flow model should show information entering from authorized users, being validated and stored in the database, being processed by flight, booking, gate, and baggage management functions, and being delivered through search results, boarding passes, notifications, and dashboard views.

B.5 Additional Recommended Diagrams

- Class Diagram — representing the core entities (User, Flight, Booking, Baggage, Gate, Payment, etc.) and their relationships.
- Activity Diagram — modeling the end-to-end passenger booking workflow (search → select seat → pay → confirm).
- Sequence Diagram — illustrating the check-in and boarding-pass generation sequence between Passenger, Ground Staff, and the system.
- Deployment Diagram — showing the frontend (Netlify/Vercel), backend (Render), and database (MySQL on Render) as deployed nodes communicating over HTTPS.

Appendix C: To Be Determined List

- TBD-01: Exact JWT expiry duration (e.g., 1 hour vs. 24 hours) to be finalized with the team. (Section 4.1, REQ-F07)
- TBD-02: Exact cancellation cutoff time before departure (e.g., 2 hours, 24 hours) to be finalized. (Section 4.3, REQ-F19)
- TBD-03: Dashboard auto-refresh interval (e.g., 30 seconds, 1 minute, or manual-only). (Section 4.8, REQ-F42)
- TBD-04: Final list of supported languages if the multi-language interface stand-out feature is implemented. (Section 6.3)
- TBD-05: Whether a physical QR/barcode scanner will be integrated at check-in counters, or whether check-in remains manual/ID-based only. (Section 3.2)